

MAGNA · SECURITY REVIEW

Magna Airdrop PR Review.

A security review of Airdrop PR Review.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Magna	Airdrop PR Review	Guardian	December 22 – December 23, 2025	December 30, 2025

§ 00

Contents.

12 pages — overview, confidence ranking, scope, 2 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 2 findings	P. 08
§ 05	Disclaimer	P. 11
§ 06	About Guardian	P. 12

§ 01

Overview.

Guardian reviewed Airdrop PR Review across a single review round. The final report includes 1 Low, 1 Informational findings.

AUDIT FIRM	REPOSITORY
Guardian	magna-eng/protocol
CLIENT	METHODOLOGY
Magna	Smart contract · Manual · Main Review
PROJECT	
Airdrop PR Review	
PREPARED BY	
Guardian	
AUDIT WINDOW	
December 22 – December 23, 2025	
FINAL REPORT	
December 30, 2025	

1.1 Confidence Ranking

5 / 5

Very High Confidence

Guardian assigns a Confidence Ranking of 5 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0
High	0	0	0	0	0
Medium	0	0	0	0	0
Low	1	0	1	0	0
Informational	1	0	1	0	0
All severities	2	0	2	0	0

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence ASSIGNED	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Airdrop PR Review in `magna-eng/protocol`.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH

Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM

Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW

Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH

Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM

Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW

Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
No files were recorded in scope.		

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
L-01	Unbounded Per-allocation Claim Fees	Validation	LOW	ACKNOWLEDGED	p. 8
I-02	getClaimFee Can't Quote Fee From allocationId	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 10

LOW **L-01**

ACKNOWLEDGED

Unbounded Per-allocation Claim Fees

CATEGORY Validation
LOCATION MerkleVester.sol
REVIEW Main Review

DESCRIPTION

The claim fee is now stored inside each allocation leaf and used directly during withdrawals without any on-chain maximum. The allocation struct includes a `claimFee` field and the withdrawal flow uses that value to compute the required ETH, so the contract will enforce whatever fee is encoded in the merkle tree. This removes the previous on-chain guardrail and shifts correctness to off-chain tooling and UI expectations, because the contract only compares `claimFee` against `msg.value` and does not validate bounds.

```
struct Allocation {  
    string id;  
    address originalBeneficiary;  
    uint256 totalAllocation;  
    bool cancelable;  
    bool revokable;  
    bool transferableByAdmin;  
    bool transferableByBeneficiary;  
    uint256 claimFee;  
    bytes extraData;  
}  
  
modifier checkFee(string memory allocationId, uint256 claimFee) {  
    uint256 expectedFee = getClaimFee(allocationId, claimFee);  
    if (expectedFee > 0) {  
        transientFeeIncurred += expectedFee;  
        require(transientFeeReceived >= transientFeeIncurred, TooSmallFeeAmountSent());  
    }  
    _;  
}  
  
function getClaimFee(string memory allocationId, uint256 claimFee) public view returns  
(uint256) {  
    if (shouldPayClaimFeeOnlyOnce && feeAlreadyPayed[allocationId]) {  
        return 0;  
    }  
}
```

L-01 Unbounded Per-allocation Claim Fees

CONTINUED

```
} else {  
  return claimFee;  
}  
}
```

Impact: Users can be economically blocked from claiming if fees are set unreasonably high and the contract provides no on-chain protection against excessive fees.

— RECOMMENDATION

Consider introducing an on-chain guardrail by enforcing a maximum fee, either as an absolute cap or as a percentage of the allocation size, and validate `claimFee` against it during withdrawal or merkle root acceptance. If the design intentionally relies on off-chain checks, add explicit governance approval for fee changes and ensure the UI and tooling reject out-of-policy `claimFee` values before roots are published.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-02

ACKNOWLEDGED

getClaimFee Can't Quote Fee From allocationId

CATEGORY Informational
LOCATION MerkleVester.sol
REVIEW Main Review

— DESCRIPTION

The getClaimFee function now requires the caller to provide a claimFee value as an input parameter. As a result, the function can no longer be used to derive the claim fee for a given allocationId by itself. This removes the ability for integrators or users to query the correct claim fee directly from the contract.

— RECOMMENDATION

Consider adding a view function that derives and returns the correct claim fee for a given allocationId, allowing callers to query the expected fee without prior knowledge.

— RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.